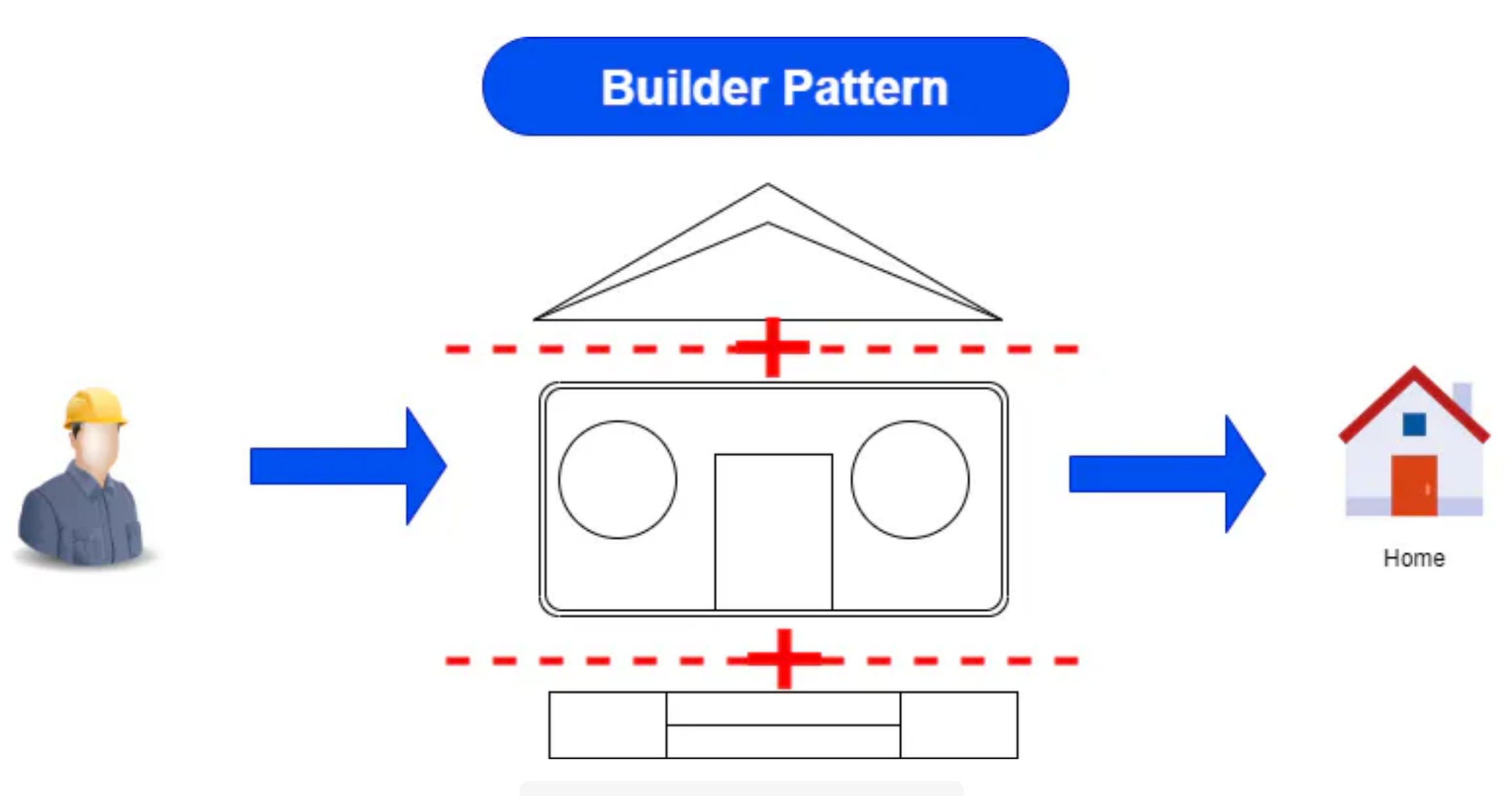
Builder Pattern

The Builder Design Pattern is a creational pattern used to construct complex objects step by step. It separates the construction of a complex object from its representation, allowing the same construction process to create different representations. This pattern is particularly useful when an object needs to be created with many possible configurations and combinations of its components.

Real-Life Example



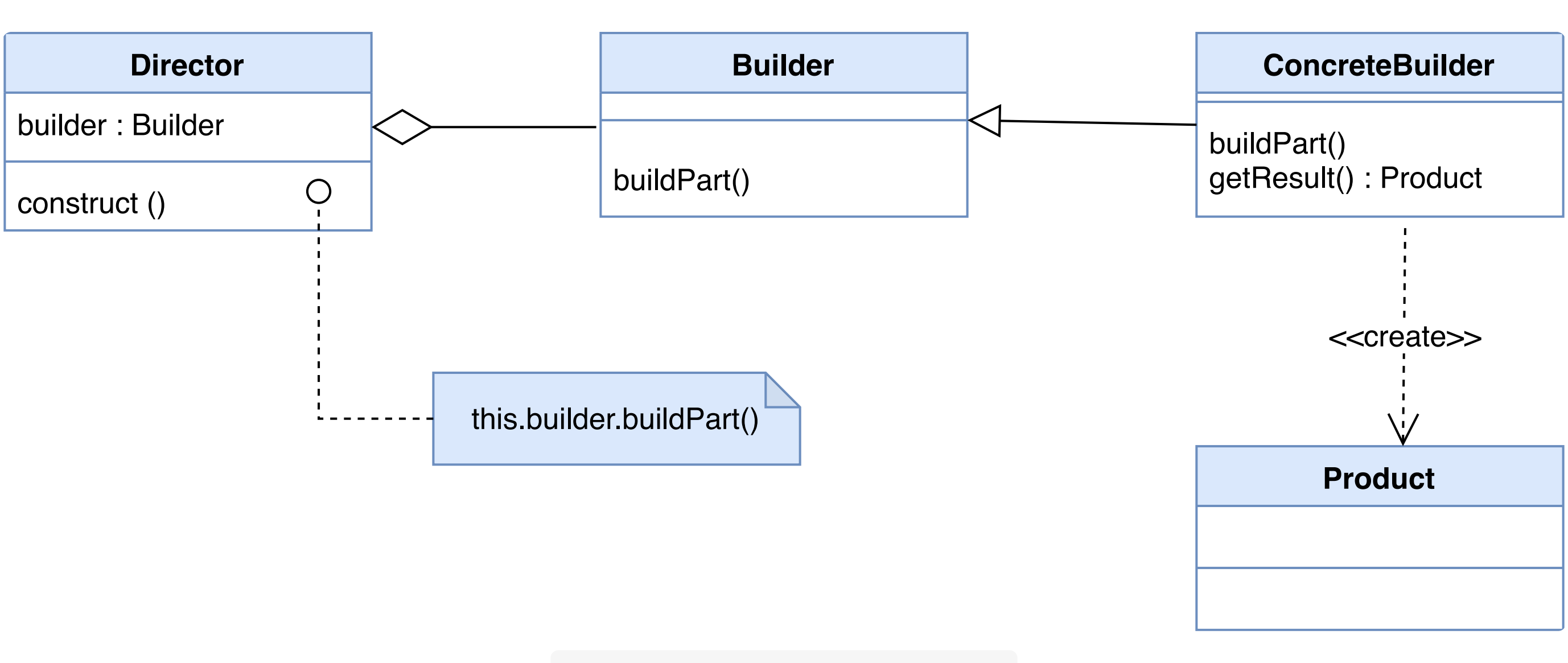
Builder Pattern - Example

To give you an idea of how this pattern works in the real world, let's consider building a house. You wouldn't start construction without a detailed plan, right? You'd need to figure out things like how many rooms you want, what type of materials to use, and how everything should be laid out. Similarly, the Builder Pattern can serve as a blueprint for your software object if it has lots of parameters, especially if many of them are optional.

Structure of Builder Pattern

The class diagram of Builder Pattern contains four types of classes:

- `Director`: Initiates the builder to get the product.
- `Builder`: Defines the blueprint for creating products.
- `ConcreteBuilder`: Provides implementation for the `Builder`, constructing and assembling parts to build the products.
- `Product`: The complex object that is being built.



Builder Pattern - Class Diagram

How It Works

- The client creates a Director object and configures it with a Builder object.
- The Director instructs the Builder on what parts to build and in what order.
- The Builder constructs the parts of the complex object and keeps track of the construction state.
- The client retrieves the final product from the Builder.

Implementation of Builder Pattern

This section will explain the implementation of Builder Pattern in four languages: C++, Java, Python, and JavaScript.

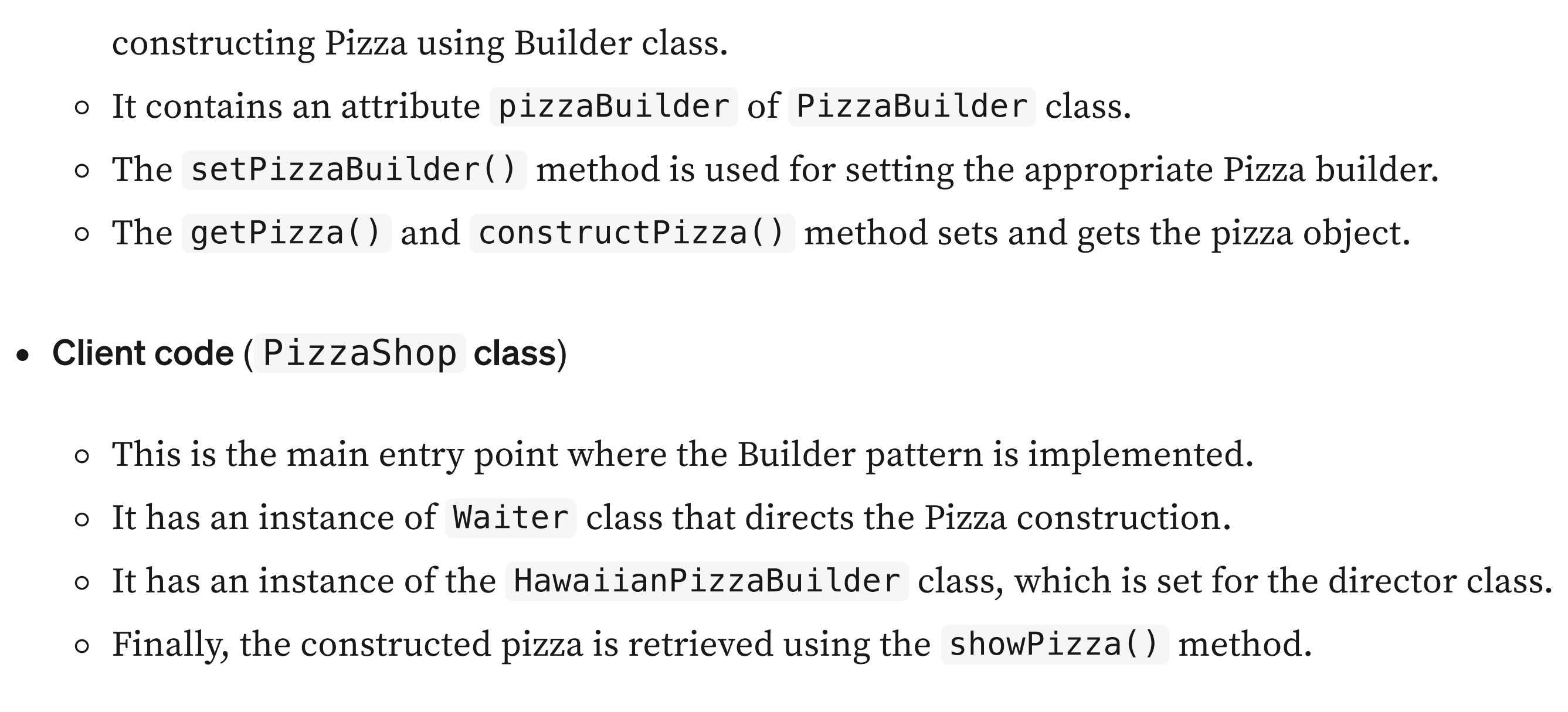
Think about a pizza class. Pizza can come in various sizes, shapes, and topping combinations. The Builder Pattern can simplify this procedure by eliminating the need for several constructors for each combination or making the user remember parameter order.

Pseudocode

Let's first discuss the pseudocode for the above-discussed Pizza example. Then we will look at the implementation of it in different languages.

```
CLASS Pizza:
    PRIVATE size, crust, toppings

    CONSTRUCTOR(size, crust, toppings):
        SET this.size = size
        SET this.crust = crust
        SET this.toppings = toppings

    METHOD setSize(size):
        SET this.size = size

    METHOD setCrust(crust):
        SET this.crust = crust

    METHOD setToppings(toppings):
        SET this.toppings = toppings

    METHOD showPizza():
        PRINT "Pizza Size:", size, ", Crust:", crust, ", Toppings:", toppings

ENDCLASS

ABSTRACT CLASS PizzaBuilder:
    PROTECTED pizza

    METHOD getPizza():
        RETURN pizza

    METHOD createNewPizzaProduct():
        SET pizza = NEW Pizza("default", "default", "default")

    ABSTRACT METHOD buildSize()
    ABSTRACT METHOD buildCrust()
    ABSTRACT METHOD buildToppings()

ENDCLASS

CLASS HawaiianPizzaBuilder EXTENDS PizzaBuilder:

    OVERRIDE METHOD buildSize():
        SET pizza.size = "Large"

    OVERRIDE METHOD buildCrust():
        SET pizza.crust = "Thin"

    OVERRIDE METHOD buildToppings():
        SET pizza.toppings = "Ham and Pineapple"

ENDCLASS

CLASS Waiter:

    PRIVATE PizzaBuilder pizzaBuilder

    METHOD setPizzaBuilder(builder):
        SET pizzaBuilder = builder

    METHOD getPizza():
        RETURN pizzaBuilder.getPizza()

    METHOD constructPizza():
        CALL pizzaBuilder.createNewPizzaProduct()
        CALL pizzaBuilder.buildSize()
        CALL pizzaBuilder.buildCrust()
        CALL pizzaBuilder.buildToppings()

ENDCLASS

MAIN:

    DECLARE waiter = NEW Waiter
    DECLARE hawaiianPizzaBuilder = NEW HawaiianPizzaBuilder

    CALL waiter.setPizzaBuilder(hawaiianPizzaBuilder)
    CALL waiter.constructPizza()

    DECLARE pizza = waiter.getPizza()
    CALL pizza.showPizza()

ENDMAIN
```

- **Product class** (`Pizza` **class**)
  - The `Pizza` class is defined as the product class having some attributes.
  - The attributes are `size`, `crust,` and `toppings`.
  - Its setter methods are also provided.
  - A function `showPizza()` displays all the attributes of the Pizza.
- **Abstract Builder** (`PizzaBuilder` **class**)
  - The `PizzaBuilder` class is defined, which represents the `Pizza` class's abstract builder.
  - It contains the protected attribute `pizza`, a member of the `Pizza` class.
  - The current pizza instance is returned by the `getPizza()` method.
  - A new pizza is initiated with default values using the `createNewPizzaProduct()` method.
  - There are three abstract methods defined: `buildSize()`, `buildCrust()`, and `buildToppings()`. Concrete builders will override these to set specific pizza properties.
- **Concrete Builder** (`HawaiianPizzaBuilder` **class**)
  - The `HawaiianPizzaBuilder` is a concrete class of `PizzaBuilder` abstract class.
  - It overrirdes the abstract methods `buildSize()`, `buildCrust()`, and `buildToppings()`
- **Director** (`Waiter` **class**)
  - This `Waiter` class serves as the director class. It is the class that is responsible for constructing Pizza using Builder class.
  - It contains an attribute `pizzaBuilder` of `PizzaBuilder` class.
  - The `setPizzaBuilder()` method is used for setting the appropriate Pizza builder.
  - The `getPizza()` and `constructPizza()` method sets and gets the pizza object.
- **Client code** (`PizzaShop` **class**)
  - This is the main entry point where the Builder pattern is implemented.
  - It has an instance of `Waiter` class that directs the Pizza construction.
  - It has an instance of the `HawaiianPizzaBuilder` class, which is set for the director class.
  - Finally, the constructed pizza is retrieved using the `showPizza()` method.



```
// The final product - Pizza
class Pizza {
    private String size;
    private String crust;
    private String toppings;

    // Constructor to initialize the pizza
    public Pizza(String size, String crust, String toppings) {
        this.size = size;
        this.crust = crust;
        this.toppings = toppings;
    }

    // Setter methods
    public void setSize(String size) {
        this.size = size;
    }

    public void setCrust(String crust) {
        this.crust = crust;
    }
```

You must run your code first

Applications of Builder Pattern

Let's discuss some of the example scenarios where Builder Pattern can help us.

- **Complex Object Creation**

  Imagine you are creating a computer configuration system. This is a complex object creation scenario. A computer comprises many different parts, including a CPU, GPU, RAM, storage, ports etc.

  Each computer's configuration can differ greatly depending on the user's needs. While some people may prioritize storage for their data demands, others may require a powerful GPU for gaming. A flexible system that builds a computer object step by step while carefully handling every configuration can be created using the Builder Pattern.

- **Producing Different Representations**

  Consider developing a document conversion system that can represent a document in several formats, such as PDF, Word, HTML, etc. The document's content remains consistent, but its presentation changes. A helpful technique that can be used in this case is the Builder Pattern. It allows you to divide the creation of the document's content from its presentation, which makes everything much more structured and easier to manage.

- **Parameter Overloaded in Constructor**

  Let's consider you're creating a system called `UserProfile`. Users have information like `name`, `age`, `email`, `address`, `hobbies`, `occupation`, and a variety of additional attributes.

  Instead of creating a constructor with multiple parameters (many of which may be unneeded), you may use the Builder Pattern. Creating an object has never been easier! We've simplified the process by allowing you to only provide the parameters you're interested in and ignore the rest. It's now more understandable than ever before.

These are some example scenarios in which Builder Pattern can be used to increase the system efficiency.

Pros and Cons

Pros	Cons
Separation of Construction and Representation: Allows you to construct complex objects step by step.	**Complexity:** Introduces multiple additional classes, which increases the complexity of the code.
Encapsulation: Encapsulates the construction logic of an object, which can simplify the client code.	**Redundancy:** For simpler objects, the builder pattern might be overkill, leading to unnecessary redundancy.
Control over Object Construction Process: The construction process can be controlled more finely than with other creation patterns.	**Specificity:** Each different type of object requires a new concrete builder.
Immutability: Can be used to build immutable objects without needing a large number of constructor parameters.	**Understanding:** Can be harder to understand and implement properly, especially for new developers.
Fluent Interfaces: Can provide a fluent API to improve readability and ease of use of object creation.	**Duplication:** Sometimes there can be duplication of code in the builders if there is overlap in the parts they assemble.

Summary

The Builder Design Pattern is ideal for constructing complex objects, especially when the construction process must allow different representations or configurations of the object. It provides clarity and flexibility in how objects are constructed and is particularly useful in cases where objects need to be created with a large number of optional components or configurations. However, greater complexity and the possibility of duplicate code come at the expense of this flexibility and control.

← Previous Next →
Abstract Factory Pattern Prototype Pattern
☐ Mark as Completed